CSCI 1101: Programming Principles I

Basic Information

- **ADA University, Fall Semester, 2026**
- **CSCI 1101: Programming Principles I (6 Credits)**
- **Course meeting times and location**

Day	Time	Room
Tuesday	16:00 – 17:15	Room B102
Friday	16:00 – 17:15	Room B101

- **Instructor:** Abbas Aliyev (PhD candidate at ADA University; MS in Computer Science / Data Analytics), Room B017
- **How to contact the instructor**
  - **Preferred mode of communication:** the course discussion forum on Blackboard. Students are expected to actively use the forum for asking their questions: answers there are visible to the whole class.
  - **Email:** ashaliyev@ada.edu.az
  - **Office hours:** by appointment only; there are no drop-in hours. Book a slot in Room B017 through Microsoft Bookings: https://outlook.office.com/bookwithme/user/0f8e94f5a2a44eedac6d5334482ed8a5@ada.edu.az/meetingtype/y67wj76S7kORMgYrhUCocQ2?anonymous&ismsaljsauthenabled&ep=mlink
- **Course web page:** Blackboard. All slides, class code, class notes, knowledge checks, practice tests, announcements and grades are distributed there.

Course Description

- **Prerequisites:** none. The course starts from the beginning: no programming experience is expected.
- **Technology requirements**
  - Bring your laptop to every class. You will be asked to write and run code during the class. Quizzes and exams are written on paper, without a laptop.
  - **Editor.** We recommend **Visual Studio Code** with the **C/C++ extension**. It is free and it works the same on Windows, macOS and Linux. You may use another editor if you prefer one.
  - **Compiler.** You may use **any C compiler** that accepts standard C. Your programs must compile and run without changes to the code. The course supports these:
    - **Linux:** GCC or Clang, installed with your distribution's package manager (for example `sudo apt install build-essential` on Ubuntu).
    - **macOS:** Clang, installed by running `xcode-select --install` in the Terminal.
    - **Windows, easiest: MinGW-w64 GCC** through the course's one-click installer `install-gcc-user.bat`. This is the option we support best.
    - **Windows, other ways: WSL** (Windows Subsystem for Linux), then GCC inside it as on Linux; or **MSVC**, the Microsoft C/C++ compiler (`cl.exe`), installed with **Visual Studio Community** or the **Build Tools for Visual Studio** and used from the Developer Command Prompt or from VS Code.

    - **Any platform, nothing to install: CoCalc** at https://cocalc.ai/, which runs in your browser.
  - **Git** is useful but not required.
  - Step-by-step instructions for each of these, with the exact commands and buttons, are in **"Setting up a C compiler"** on Blackboard. Follow that guide first; bring any problem to the forum.
- **Overview of the course:** this is a first course in programming, taught in the C language. It starts from the first program and builds up one idea at a time. **Part I** (before the midterm) covers the core of the language: variables, input and output, decisions, loops, functions and recursion, arrays, strings and two-dimensional arrays. **Part II** (after the midterm) looks under the surface: memory and pointers, command-line arguments, structures, enumerations and unions, dynamic memory, files, and searching and sorting. Every idea is shown on small programs that are written and run in class, and the code of every class is posted so that students can run it and change it themselves. Three programming projects apply the ideas to larger problems.
- **Student learning objectives:** after this course, students are expected to be able to:
  1. Write, compile, run, test and debug C programs from the command line.
  2. Use variables, the fundamental data types, input and output, decisions and loops.
  3. Decompose a larger problem into functions, and explain variable scope, call-by-value and recursion.
  4. Work with arrays, strings and multidimensional arrays.
  5. Use pointers, structures and dynamic memory allocation.
  6. Read, write and process files.
  7. Apply basic searching and sorting techniques, and analyze a program to predict what it does.
- **Methods of instruction:** two 75-minute classes per week. The class will be taught through lectures, including discussions around class examples and course homework. Discussions based on student contributions add a vital and dynamic element to the class. Students are expected to come to the class with comments or questions from the course readings. Eight short in-class quizzes check understanding as the course goes along. After every class a short online knowledge check opens on Blackboard, so that you can test yourself on that class's material at no cost to your grade.
- **Workload:** It is estimated that the students will need to spend 3-4 hours studying and preparation for the classes every week. Estimated amount of time to spend on course homework is additional 3-4 hours per week.

Materials

- **Slides:** posted on Blackboard for every class. The slides are the primary material and define what is examinable. No textbook is required.
- **Class code:** every program shown or written in class is posted on Blackboard, with a note at the top of each file on what it does, how to run it, and what to try next.
- **Class notes:** a written version of each class, in plain words, posted on Blackboard for reading at home.
- **Setting up a C compiler:** the step-by-step guide on Blackboard (see Technology requirements).
- **Practice tests:** on Blackboard, not graded. A practice test opens after its knowledge check, with unlimited attempts and feedback on every answer.

Policies

- **Grading procedures**
  - Students are graded on an absolute scale.
  - The course grade is calculated from the following components:

Quizzes	Project 1	Project 2	Project 3	Midterm exam	Final exam
15%	10%	10%	15%	20%	30%

  - **Grade mapping (ADA defaults):**

A+	A	A−	B+	B	B−	C+	C	C−	D+	D
98.0	93.5	89.5	86.5	82.5	79.5	76.5	72.5	69.5	66.5	59.5

- **Attendance and tardiness:** attendance is an indispensable element of the educational process. In compliance with Azerbaijani legislation, instructors are required to monitor attendance and inform the Registrar and the Dean of the respective School when students miss significant amounts of class time. Azerbaijani legislation mandates that students who fail to attend at least 75% of classes will fail the course.
  - Starting from the second week of the semester (right after the add/drop period ends on 16 September 2026), attendance is checked before the start of each class. There is no late attendance mark.
  - Quizzes are written in class, at the start of a class (usually a Friday class), and cannot be taken remotely or made up.
  - In case of illness or other emergency, students are expected to submit documented proof to their advisors. Advisors then will forward the document to the students' relevant instructors, who will decide whether to accept the absence as excused or not.
- **Missed or late assignments/extensions:** Deadlines will never be extended. When submitting, a student must consider possible internet connection issues, uploaded file size, etc. Late submissions of one minute until thirty minutes will receive a 10% penalty, late submissions of thirty minutes until one day will receive a 25% penalty with no claim for bonuses (if there are any). For each next day, a further 25% penalty will be applied.
- **Standards for academic honesty and penalties for infractions:** a student found guilty of academic dishonesty for the first time after the "Forgiveness Period" fails the course. Repeated violations lead to suspension or permanent expulsion. For more information, please read the Honor Code.
- **Accommodation:** ADA University provides appropriate academic accommodations for qualified students with documented disabilities. Any student who may need an accommodation should notify the Office of Disability Services and Inclusive Education before the start of the academic term, and is welcome to discuss arrangements with the instructor.

Requirements

- **Quizzes (15%):** Eight short quizzes will be conducted throughout the semester, usually on Fridays; the Schedule below gives their dates. Each quiz will contain only one or two short questions and will focus on fundamental concepts recently covered in class. The quizzes are not intended to be difficult; their purpose is to encourage continuous study and verify students' understanding of the basic programming concepts. **Only the best six quiz grades will be included in the final grade.** Therefore, no make-up quizzes will be provided. The number of quizzes was changed from six (best five) to eight (best six) by a vote of the students of this section, held on 20–24 September 2026.
- **Knowledge checks (0%):** after every class a short online knowledge check opens on Blackboard: about 6 multiple-choice questions on that class's material, drawn from a small pool, so the same question may come up again in another attempt. Each check stays open for **three weeks**. You have **three attempts** and the best one is kept. The checks carry **no grade weight** and **cannot lower your mark**. They exist so that you can find out what you did not understand while

there is still time to ask. Each check you complete earns a Blackboard badge. They are the best preparation for the quizzes, which cover the same material without a laptop. Students who complete every check on time may receive small bonuses at the instructors' discretion.

- **Programming projects (35%: 10%, 10% and 15%):** Students will complete three programming projects during the semester. The projects will require students to apply the programming concepts learned in class to larger and more practical problems. Projects will gradually increase in complexity as new topics are introduced. Each student is responsible for understanding all parts of the code submitted as part of a project. After submission, students may be asked by the Teaching Assistant to explain selected parts of their implementation, describe how particular functions or algorithms work, or make a small modification to their program. This verification is considered part of the project assessment. A student who is unable to demonstrate sufficient understanding of the submitted work may receive a reduced project grade.
- **Midterm exam (20%):** on **Friday 30 October 2026**, in the time of class 16; the room will be announced. It covers classes 1–14. The midterm examination will be conducted in a paper-based format and will assess the topics covered during the first part of the semester. Questions may include conceptual questions, code tracing, predicting program output, identifying and correcting errors, completing incomplete code, and writing short C programs or functions.
- **Final exam (30%):** in the final exam period, 22 December 2026 to 6 January 2027 (date set by the Registrar). The final examination will also be paper-based and will be cumulative, with greater emphasis on the topics covered during the second half of the course. Like the midterm, students may be asked to analyze C code, determine program output, identify errors, explain programming concepts, and write or complete short programs and functions. The final examination is intended to evaluate each student's individual understanding of the programming principles covered throughout the course.

Schedule

- **Last day to add/drop:** 16 September 2026
- **Last day to withdraw from the course:** 14 November 2026
- **No classes:** 9 and 10 November 2026 (public holidays).
- **Last day of classes:** 19 December 2026. **Exam period:** 22 December 2026 to 6 January 2027, with a New Year break from 31 December to 4 January.
- Square brackets in the Assessment column give the classes a quiz or exam covers. For example, *Quiz #2 [3–6]* covers classes 3 to 6. A quiz is written in the first 25 minutes of its class.
- Knowledge checks are not listed in the table: **every class has one**, and it closes about three weeks after that class.
- The table has one row for every class. Slides are posted on Blackboard for each class.
- Project dates are given by week here. The exact dates come with each project.

Part I — From the first program to arrays and strings

#	Date	Topic	Learning outcomes	Assessment
1	Tue 8 Sep	Course intro. Setting up the editor and the compiler. The first C program.	Set up the programming environment; use the command line to compile and run a program.	
2	Fri 11 Sep	Variables, `printf`, `scanf` and math functions	Declare and use variables; read input and print output; use arithmetic and the math functions.	

#	Date	Topic	Learning outcomes	Assessment
3	Tue 15 Sep	Data types, comparisons and `if`	Use the fundamental C data types; compare values; write `if` and `if`/`else`.	
4	Fri 18 Sep	`else if`, `switch` and worked examples	Choose between several paths with `else if` and `switch`; read and write small programs that make decisions.	Quiz #1 [1–2]
5	Tue 22 Sep	Loops: `while` and `for`	Write `while` and `for` loops; update counters with `++`, `--` and `+=`; trace a loop by hand.	
6	Fri 25 Sep	Loop patterns, tracing and practice	Keep a running total or a count in a loop; check input with a loop; choose where a loop starts and stops; use `break` and `continue`.	Project #1 announced this week
7	Tue 29 Sep	Functions: defining and calling	Define a function with parameters and a return value; call it; split a program into functions.	
8	Fri 2 Oct	Variable scope. Call-by-value. Recursion.	Explain where a variable can be used; explain why a function cannot change its caller's variables; solve simple problems recursively.	Quiz #2 [3–6]
9	Tue 6 Oct	Linear arrays	Declare a one-dimensional array; fill it and print it with a loop.	
10	Fri 9 Oct	Working with arrays: search, min/max, bubble sort	Search an array; find its smallest and largest value; sort it with bubble sort.	Project #1 deadline this week
11	Tue 13 Oct	Characters and strings	Work with `char` values; store, read and print strings.	
12	Fri 16 Oct	String manipulation. Integer-string conversions.	Perform common string operations; convert between numbers and strings.	
13	Tue 20 Oct	Multidimensional arrays	Declare a two-dimensional array; process it with nested loops.	Quiz #3 [7–10] (moved from Fri 16 Oct). Project #2 announced
14	Fri 23 Oct	Working with tables and matrices	Solve problems with tabular and matrix-like data.	Quiz #4 [11–12]
15	Tue 27 Oct	Writing large programs: C header files. Variable arguments.	Organize a larger C program with header files; use functions with a variable number of arguments.	
16	Fri 30 Oct	Midterm exam	Show on paper what you learned in classes 1–14.	Midterm [1–14], in this class's time; the room will be announced

Part II — Memory, pointers, structures and files

#	Date	Topic	Learning outcomes	Assessment
17	Tue 3 Nov	Intro to memory. Pointers.	Explain how variables sit in memory; declare and use pointers.	
18	Fri 6 Nov	Pointers and functions: pass-by-reference	Change a caller's data through a pointer; write functions that take pointers.	Project #2 deadline Mon 9 Nov, 23:59
—	Tue 10 Nov	*No class (public holidays 9–10 Nov)*		

#	Date	Topic	Learning outcomes	Assessment
19	Fri 13 Nov	Command-line arguments. Structs in C.	Use command-line arguments; define a structure and make variables of it.	Quiz #5 [17–18]
20	Tue 17 Nov	Struct field operations	Access and modify structure fields; use arrays of structures and structures in functions.	
21	Fri 20 Nov	Enums. Unions.	Define and use enumerations and unions.	Project #3 announced this week
22	Tue 24 Nov	Advanced uses of pointers	Apply pointer arithmetic; relate pointers and arrays.	
23	Fri 27 Nov	Dynamic memory allocation. Strings again.	Allocate and release memory dynamically; manipulate strings using pointers.	Quiz #6 [19–21]
24	Tue 1 Dec	File I/O: open, read, write, append	Read from and write to files; append to a file.	
25	Fri 4 Dec	File I/O: copy, merge, search in files	Copy and merge files; search file contents.	Quiz #7 [22–23]
26	Tue 8 Dec	Advanced topics: sorting	Apply sorting techniques to solve programming problems.	
27	Fri 11 Dec	Advanced topics: searching. Putting it together.	Apply searching techniques; integrate previously learned C concepts.	Quiz #8 [24–25]. Project #3 deadline this week
28	Tue 15 Dec	Final review I: the first half of the course	Review and consolidate variables, decisions, loops, functions, arrays and strings.	
29	Fri 18 Dec	Final review II: the second half of the course	Review and consolidate pointers, structures, dynamic memory and files through problems in exam format.	Last class
—	22 Dec – 6 Jan	Final exam (date set by the Registrar)		Final [1–29]

Disclaimer

- The course schedule is subject to change as necessary. Any change is announced on Blackboard.